

Experiment 4: Time and Space Complexity Comparison

Objective: Compare Linear Search, Binary Search, Bubble Sort, Insertion Sort, Merge Sort and Quick Sort using random input data, measure execution time, and visualize theoretical time-complexity growth using Matplotlib.

1. Time and Space Complexity

Algorithm	Best Time	Average Time	Worst Time	Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ avg.

2. C Program — Execution Time

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 1000

void bubbleSort(int a[], int n) {
    for (int i=0; i<n-1; i++)
        for (int j=0; j<n-i-1; j++)
            if (a[j]>a[j+1]) {
                int t=a[j]; a[j]=a[j+1]; a[j+1]=t;
            }
}

void insertionSort(int a[], int n) {
    for (int i=1; i<n; i++) {
        int key=a[i], j=i-1;
        while (j>=0 && a[j]>key) {
            a[j+1]=a[j];
            j--;
        }
        a[j+1]=key;
    }
}

void merge(int a[], int l, int m, int r) {
    int temp[N], i=l, j=m+1, k=l;
    while (i<=m && j<=r)
        temp[k++]=(a[i]<a[j]) ? a[i++] : a[j++];
    while (i<=m) temp[k++]=a[i++];
    while (j<=r) temp[k++]=a[j++];
    for (i=l; i<=r; i++) a[i]=temp[i];
}

void mergeSort(int a[], int l, int r) {
    if (l<r) {
        int m=(l+r)/2;
        mergeSort(a, l, m);
        mergeSort(a, m+1, r);
        merge(a, l, m, r);
    }
}

int partition(int a[], int l, int r) {
    int pivot=a[r], i=l-1;
    for (int j=l; j<r; j++)
        if (a[j]<pivot) {
            i++;
            int t=a[i]; a[i]=a[j]; a[j]=t;
        }
    int t=a[i+1]; a[i+1]=a[r]; a[r]=t;
    return i+1;
}

void quickSort(int a[], int l, int r) {
    if (l<r) {
        int p=partition(a, l, r);
        quickSort(a, l, p-1);
        quickSort(a, p+1, r);
    }
}
```

```

}

int linearSearch(int a[], int n, int key) {
    for (int i=0;i<n;i++)
        if (a[i]==key) return i;
    return -1;
}

int binarySearch(int a[], int n, int key) {
    int l=0,r=n-1;
    while (l<=r) {
        int m=(l+r)/2;
        if (a[m]==key) return m;
        if (a[m]<key) l=m+1;
        else r=m-1;
    }
    return -1;
}

void copyArray(int source[], int destination[], int n) {
    for (int i=0;i<n;i++) destination[i]=source[i];
}

int main() {
    int original[N], a[N], key;
    clock_t start,end;

    srand((unsigned)time(NULL));

    for (int i=0;i<N;i++) original[i]=rand()%10000;
    key=original[N-1];

    printf("Input size: %d\n\n",N);

    start=clock();
    linearSearch(original,N,key);
    end=clock();
    printf("Linear Search : %.8f seconds\n",
        (double)(end-start)/CLOCKS_PER_SEC);

    copyArray(original,a,N);
    mergeSort(a,0,N-1);

    start=clock();
    binarySearch(a,N,key);
    end=clock();
    printf("Binary Search : %.8f seconds\n",
        (double)(end-start)/CLOCKS_PER_SEC);

    copyArray(original,a,N);
    start=clock();
    bubbleSort(a,N);
    end=clock();
    printf("Bubble Sort : %.8f seconds\n",
        (double)(end-start)/CLOCKS_PER_SEC);

    copyArray(original,a,N);
    start=clock();
    insertionSort(a,N);
    end=clock();
    printf("Insertion Sort : %.8f seconds\n",
        (double)(end-start)/CLOCKS_PER_SEC);

    copyArray(original,a,N);
    start=clock();
    mergeSort(a,0,N-1);
    end=clock();
    printf("Merge Sort : %.8f seconds\n",
        (double)(end-start)/CLOCKS_PER_SEC);

    copyArray(original,a,N);
    start=clock();
    quickSort(a,0,N-1);
    end=clock();
    printf("Quick Sort : %.8f seconds\n",
        (double)(end-start)/CLOCKS_PER_SEC);

    return 0;
}

```

3. Python Matplotlib Program — Complexity Graph

```
import numpy as np
import matplotlib.pyplot as plt

n = np.arange(1, 1001)

linear = n
binary = np.log2(n)
bubble = n**2
insertion = n**2
merge = n * np.log2(n)
quick = n * np.log2(n)

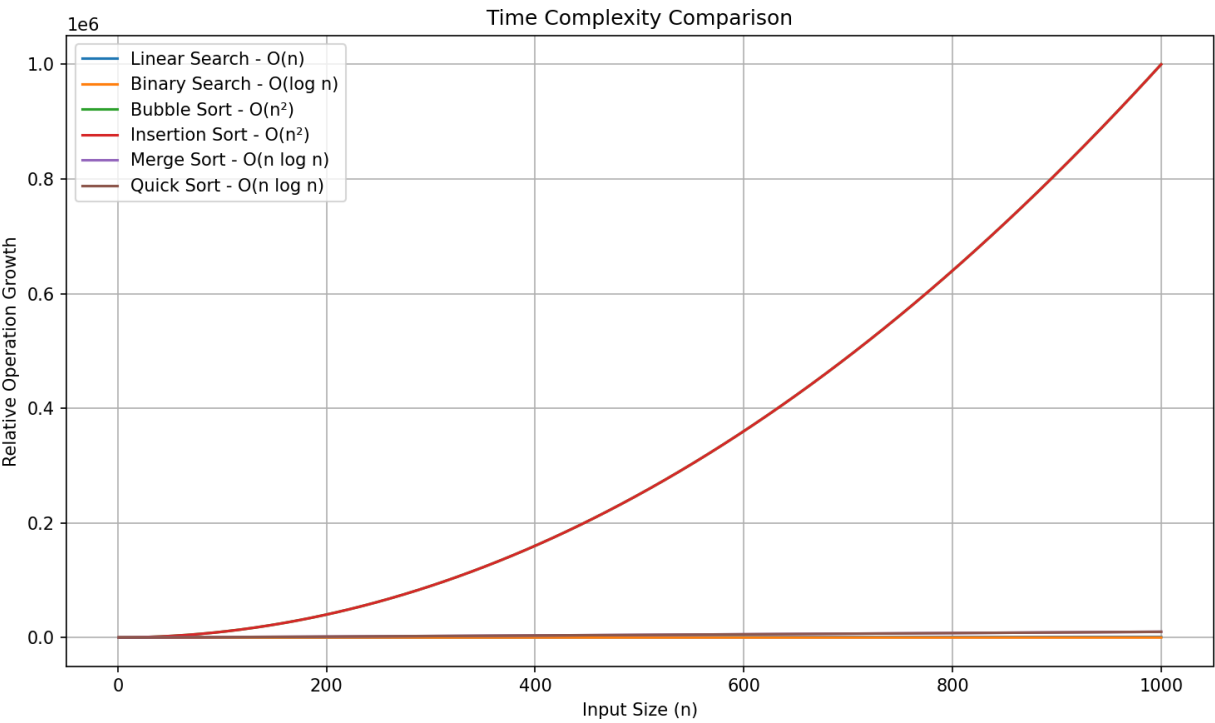
plt.figure(figsize=(10, 6))
plt.plot(n, linear, label="Linear Search -  $O(n)$ ")
plt.plot(n, binary, label="Binary Search -  $O(\log n)$ ")
plt.plot(n, bubble, label="Bubble Sort -  $O(n^2)$ ")
plt.plot(n, insertion, label="Insertion Sort -  $O(n^2)$ ")
plt.plot(n, merge, label="Merge Sort -  $O(n \log n)$ ")
plt.plot(n, quick, label="Quick Sort -  $O(n \log n)$ ")

plt.xlabel("Input Size (n)")
plt.ylabel("Relative Operation Growth")
plt.title("Time Complexity Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

4. Conclusion

Binary Search has the lowest asymptotic search growth at $O(\log n)$, provided the array is sorted. Merge Sort and average-case Quick Sort have $O(n \log n)$ time and are generally more efficient for large inputs than Bubble Sort and Insertion Sort, which have $O(n^2)$ average time. The C program measures actual execution time on the computer, while the Python graph represents theoretical growth of the algorithms.

Time Complexity Comparison Graph (Linear Scale)



Time Complexity Comparison Graph (Log Scale)

